Quicksort

Original Hoare partition scheme for quicksort.
(this is the **Hoare-style / two-pointer Quick Sort with pivot = first element**).

✅ 🔷 Algorithm: QUICK SORT

Algorithm: QUICKSORT(A, low, high)

Step 1: If low < high, then
a. pos = PARTITION(A, low, high)
b. QUICKSORT(A, low, pos - 1)
c. QUICKSORT(A, pos + 1, high)

Step 2: Exit

✅ 🔷 Algorithm: PARTITION

Algorithm: PARTITION(A, low, high)

Step 1:
pivot = low *(first element as pivot)*
left = low
right = high

Step 2: Repeat while left < right

(a) Move LEFT pointer

- Increase left until
  A[left] > A[pivot]
  or left > high

(b) Move RIGHT pointer

- Decrease right until
  A[right] <= A[pivot]
  or right < low

(c) Swap condition

- If left < right
  Swap A[left] and A[right]

Step 3:
Swap A[pivot] and A[right]

Step 4:
Return right *(final pivot position)*

Key Idea

- 👉 Left finds **greater element**
- 👉 Right finds **smaller element**
- 👉 Swap them
- 👉 Finally place pivot at correct position

👉 *Quick Sort partitions the array by placing the pivot at its correct position using two pointers, and recursively sorts the left and right subarrays.*

First Example

0	1	2	3	Array Index
23	**12**	**25**	**5**	Remarks
Pivot Left			Right	Increase Left until we find a number greater than Pivot or Left reaches extreme right. Decrease Right until we find number less than or equal to Pivot
		Left	Right	Since Left < Right Swap Left and Right
23	12	5	25	
		Right	Left	Since Left >= Right Swap Pivot and Right
(5	12)	(23)	(25)	
Pivot Left	Right			
Right	Left			Since Left >= Right Swap Pivot and Right
(5)	(12)	(23)	(25)	
	Pivot Left Right			Since Left >= Right Swap Pivot and Right

(5)	(12)	(23)	(25)	
			Pivot Left Right	Since Left >= Right Swap Pivot and Right
(5)	(12)	(23)	(25)	Sorted Data

Second Example

0	1	2	3	Array Index
27	**23**	**8**	**8**	Remarks
Pivot Left			Right	Increase Left until we find a number greater than Pivot or Left reaches extreme right. Decrease Right until we find number less than or equal to Pivot
			Left Right	Since Left >= Right Swap Pivot and Right
(8	23	8)	(27)	
Pivot Left		Right		
	Left	Right		Since Left < Right Swap Left and Right
8	8	23	(27)	
	Right	Left		Since Left >= Right Swap Pivot and Right
(8)	(8)	(23)	(27)	
Pivot Left Right				Since Left >= Right Swap Pivot and Right
(8)	(8)	(23)	(27)	
		Pivot Left Right		Since Left >= Right Swap Pivot and Right
(8)	(8)	(23)	(27)	Sorted Data

1	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*Remarks*
38	08	16	06	79	57	24	56	02	58	04	70	45	
pivot				up						down			swap up & down
pivot				04						79			
pivot					up			down					swap up & down
pivot					02			57					
pivot						down	up						swap pivot & down
(24	08	16	06	04	02)	**38**	(56	57	58	79	70	45)	
pivot					down	up							swap pivot & down
(02	08	16	06	04)	**24**								
pivot, down	up												swap pivot & down
02	(08	16	06	04)									
	pivot	up		down									swap up & down
	pivot	04		16									
	pivot		down	Up									
	(06	04)	**08**	(16)									swap pivot & down
	pivot	down	up										
	(04)	**06**											swap pivot & down
	04 pivot, down, up												
				16 pivot, down, up									
(02	**04**	**06**	**08**	**16**	**24)**	38							

							(56	57	58	79	70	45)	
							pivot	up				down	swap up & down
							pivot	45				57	
							pivot	down	up				swap pivot & down
							(45)	**56**	(58	79	70	57)	
							45 pivot, down, up						swap pivot & down
									(58 pivot	79 up	70	57) down	swap up & down
										57		79	
										down	up		
									(57)	**58**	(70	79)	swap pivot & down
									57 pivot, down, up				
											(70	79)	
											pivot, down	up	swap pivot & down
											70		
												79 pivot, down, up	
							(45	56	57	58	70	79)	
02	**04**	**06**	**08**	**16**	**24**	**38**	**45**	**56**	**57**	**58**	**70**	**79**	

Case	Time Complexity
Worst-case performance	$O(n^2)$ The worst-case complexity of Quick Sort is O(n^2), which occurs when the pivot element is either the smallest or largest value in every sub-array. This situation arises when the pivot is always the smallest or

	largest value in multiple steps of recursion, leading to a recursion depth of O(n). In such cases, each partitioning operation takes O(n) time, and the recursion depth is O(n), resulting in a total time complexity of O(n^2).
Best-case performance	O(n log n)
Average-case performance	O(n log n)
Space Complexity of Quick Sort: **Worst-case scenario: O(n)** due to unbalanced partitioning leading to a skewed recursion tree requiring a call stack of size O(n). **Best-case scenario: O(log n)** as a result of balanced partitioning leading to a balanced recursion tree with a call stack of size O(log n).	

Quicksort Program

```
while (left < right)
      {
/*Increase left until an element greater than pivot is found*/ while
        (array[left] <= array[pivot] && left <= high)
        {
          left++;
        }

/*Decrease right until an element less than or equal to pivot is found*/ while
        (array[right] > array[pivot] && right >= low)
        {
          right--;
        }

   /*
if left<right, then swap array[left] and  array[right]
   */
        if (left < right)
        {
          temp = array[left]; array[left]
          = array[right]; array[right] =
          temp;
        }
      }
      /*
if left==right or left>right than swap array[right] and array[pivot]
      */
      temp = array[right];
      array[right] = array[pivot];
      array[pivot] = temp;
      quicksort(array, low, right - 1);  // Apply quicksort on the left side of pivot quicksort(array,
      right + 1, high); // Apply quicksort on the right side of pivot
   }
}
```

Another technique of implementing Quicksort algorithm. Any element can be chosen pivot either the leftmost or the rightmost. The given program implements Quick Sort using a **modified Hoare partition scheme**. It uses two pointers (left and right) and a pivot that moves during swapping.

Quick Sort

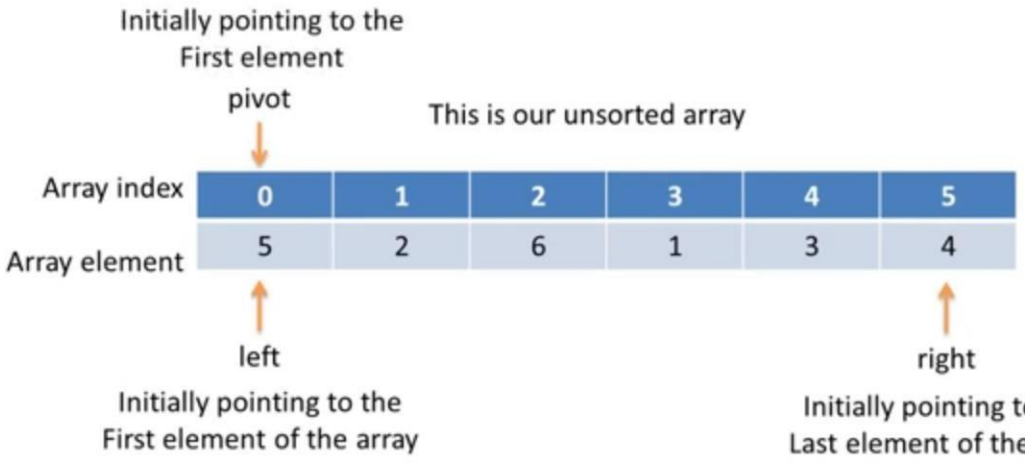
This sorting algorithm uses the idea of divide and conquer.

*It finds the element called **pivot** which divides the array into two halves in such a way that elements in the left half are smaller than pivot and elements in the right half are greater than pivot.*

Quick Sort

We follow three steps recursively:

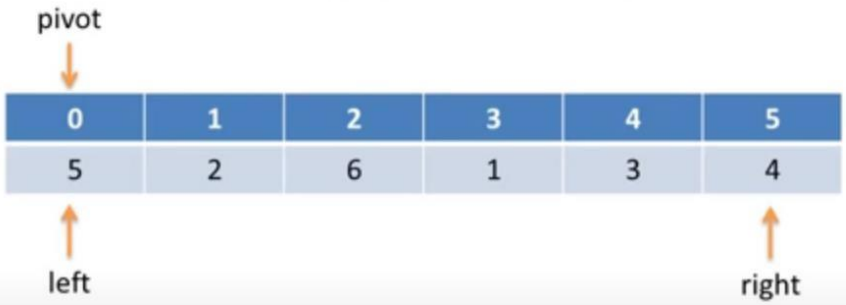
1. *Find pivot that divides the array into two halves.*
2. *Quick sort the left half.*
3. *Quick sort the right half.*

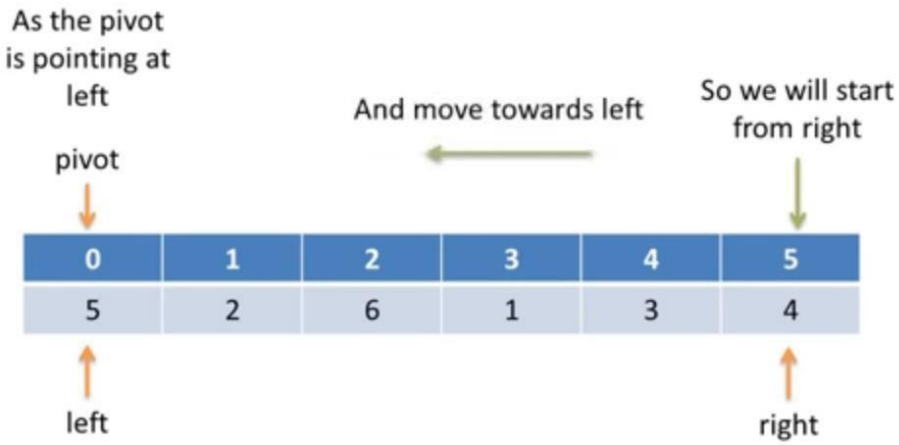


Remember this rule:

All elements to the **right** of pivot must be **greater** than pivot.

All elements to the **left** of pivot must be **smaller** than pivot.





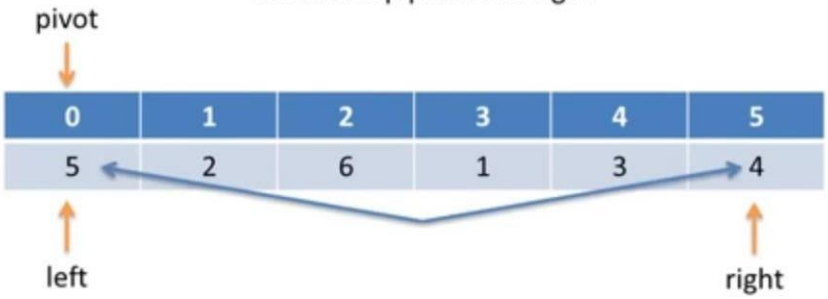
Is pivot < right?

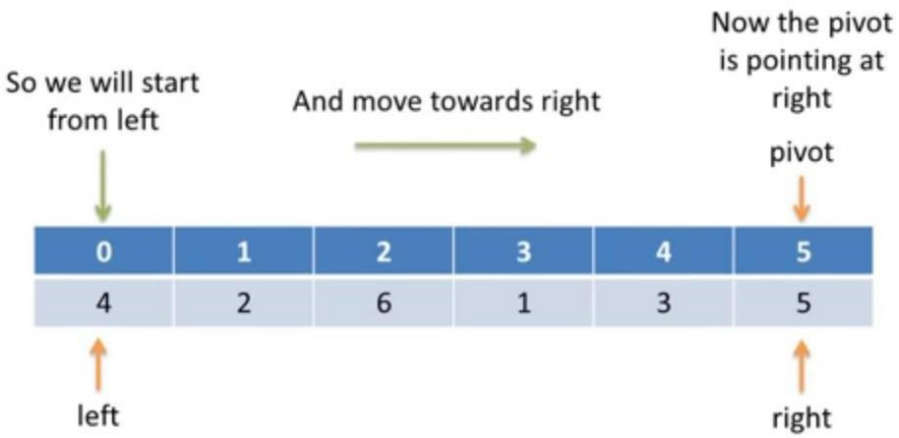
NO

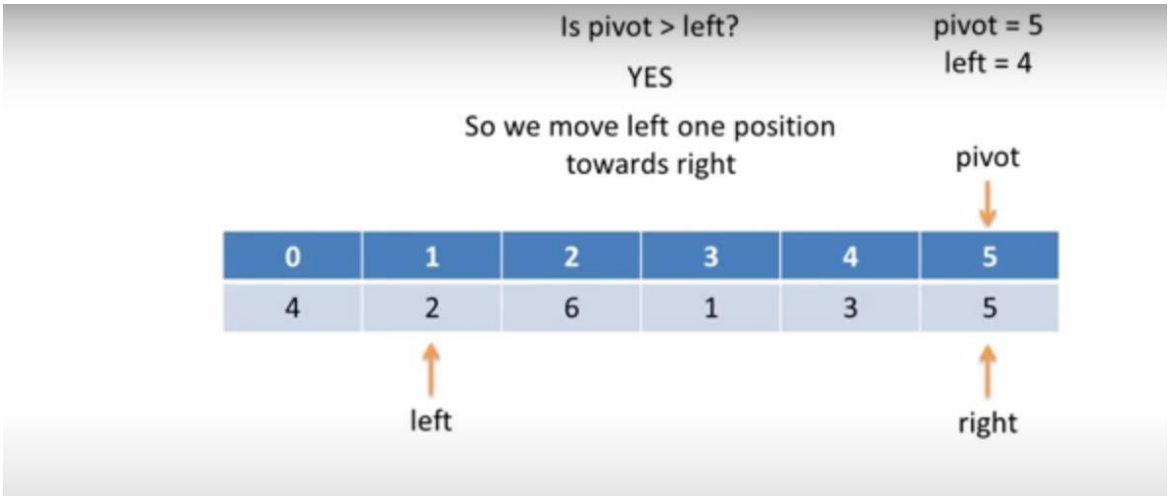
So we swap pivot and right

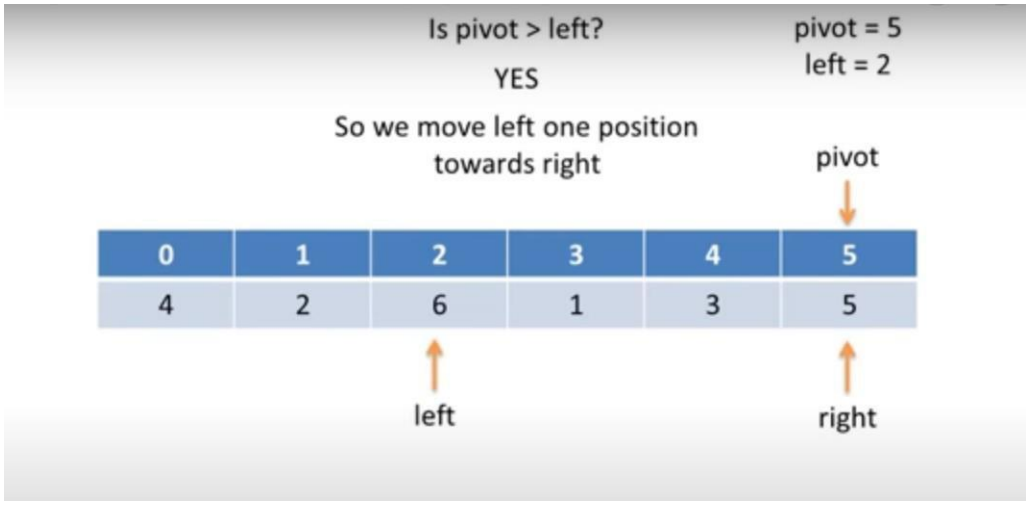
pivot = 5

right = 4




So we will start from left
And move towards right
Now the pivot is pointing at right
pivot
0 1 2 3 4 5
4 2 6 1 3 5
left
right

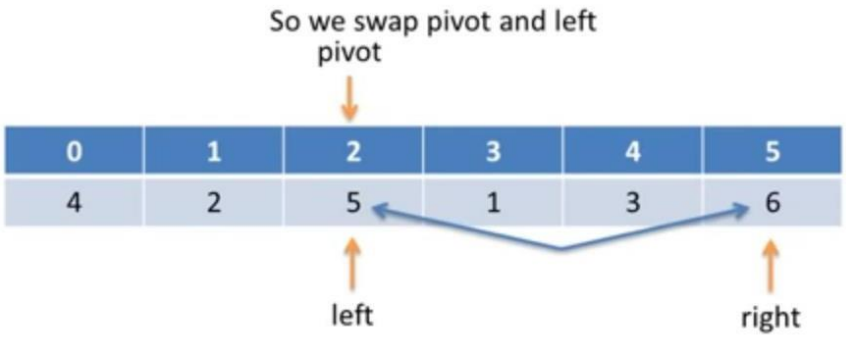

Is pivot > left?
YES
pivot = 5
left = 4
So we move left one position towards right
pivot
0 1 2 3 4 5
4 2 6 1 3 5
left
right


Is pivot > left?
YES
pivot = 5
left = 2
So we move left one position towards right
pivot
0 1 2 3 4 5
4 2 6 1 3 5
left
right

Is pivot > left?

NO

So we swap pivot and left



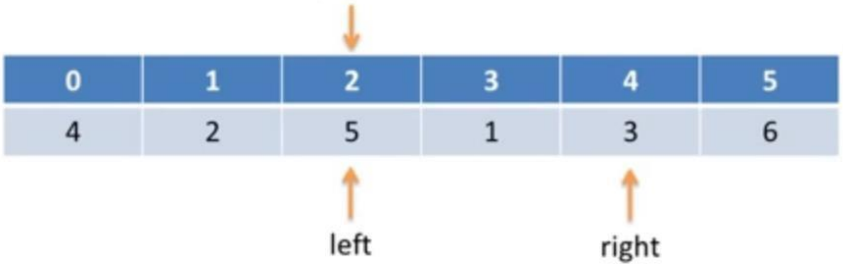
And move the pivot to left

Is pivot < right?

pivot = 5
right = 6

YES

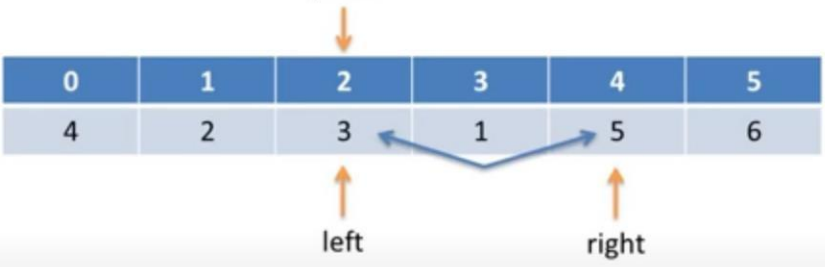
So we move right one position towards left



Is pivot < right?

NO

So we swap pivot and right



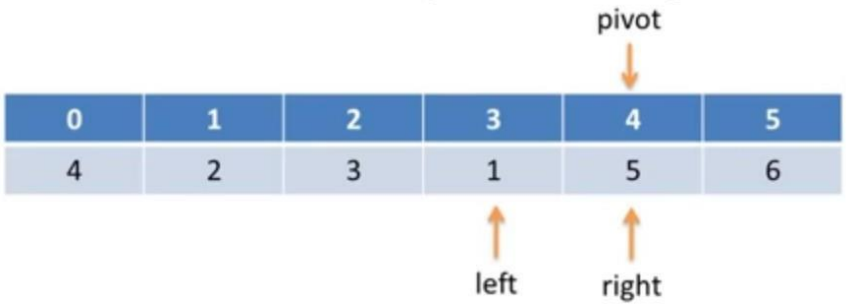
And move the pivot to right

Is pivot > left?

YES

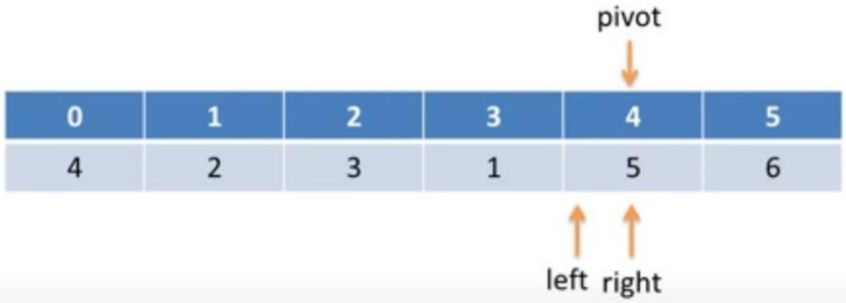
pivot = 5
left = 3

So we move left one position towards right



Now both left and right are pointing at the same element of the array

This means 5 is the pivot and it is at the sorted position.

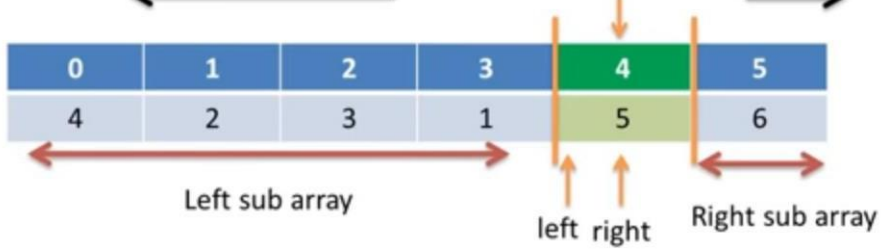


Now both left and right are pointing at the same element of the array

This means 5 is the pivot and it is at the sorted position.

Elements left of pivot are smaller

Elements right of pivot are greater



So pivot has divided the array into two sub arrays.

We will now quick sort the left sub array.

Algorithm

```
/*a[0:n-1] is an array of n elements.
beg = first index of array
end = last index of array
*/
QuickSort(a, beg, end)
Begin
   if beg < end then
      Call PartitionArray(a, beg, end, pivotLoc);    //this will find pivot location
      Call QuickSort(a, beg, pivotLoc - 1 );  //Quick sort left sub array
      Call QuickSort(a, pivotLoc + 1, end);    //Quick sort right sub array
   endif
End
```

(This is the two-pointer / moving pivot Quick Sort – modified Hoare style)

✅ 🔷 **Algorithm: QUICK SORT**

QUICKSORT(A, beg, end)
Step 1: If beg < end, then
a. Call PARTITION(A, beg, end, pivotLoc)
b. QUICKSORT(A, beg, pivotLoc - 1) *(left subarray)*
c. QUICKSORT(A, pivotLoc + 1, end) *(right subarray)*
Step 2: Exit

✅ 🔷 **Algorithm: PARTITION (Moving Pivot Method)**
PARTITION(A, beg, end, pivotLoc)
Step 1:
left = beg
right = end
pivotLoc = beg *(pivot = first element)*

Step 2: Repeat

🔶 **(a) Move RIGHT pointer**

- Decrease right until:
  - A[right] < A[pivotLoc]
    OR
  - right == pivotLoc

🔶 **(b) Check**

- If pivotLoc == right → go to Step 5
- Else
  Swap A[pivotLoc] and A[right]
  Set pivotLoc = right

🔶 **(c) Move LEFT pointer**

- Increase left until:
  - A[left] > A[pivotLoc]
    OR
  - left == pivotLoc

🔶 **(d) Check**

- If pivotLoc == left → go to Step 5
- Else
  Swap A[pivotLoc] and A[left]
  Set pivotLoc = left

Step 3: Repeat Step 2

Step 4: (Loop continues until pivot reaches correct position)

Step 5:
Return pivotLoc

Key Idea

👉 Right finds **smaller element → swap with pivot**
👉 Left finds **larger element → swap with pivot**
👉 Pivot keeps **moving left and right**
👉 Stops when pivot reaches correct position

👉 *This Quick Sort uses a moving pivot and two pointers (left and right) to place the pivot in its correct position, then recursively sorts the left and right subarrays.*

```
// Quick Sort in C
#include <stdio.h>
// Function declarations
/*
 a → passed as pointer so the original array can be modified (in-place sorting)
 pivotLoc → passed as pointer so the function can return the pivot index
```

```
 (C allows returning multiple values using pointers / call-by-reference)
*/
void partitionArray(int *a, int beg, int end, int *pivotLoc);
void quickSort(int *a, int beg, int end);
int main()
{
   // Unsorted array
   int a[] = {5, 2, 6, 1, 3, 4, 2, 5, 7};
   // Calculate number of elements
   int n = sizeof(a) / sizeof(a[0]);
   int i;
   // Call quickSort on entire array
   quickSort(a, 0, n - 1);
   // Print sorted array
   printf("Sorted Array:\n");
   for(i = 0; i < n; i++)
   {
      printf("%d\t", a[i]);
   }
   return 0;
}//end of main()

// Quick Sort Function
void quickSort(int *a, int beg, int end)
{
   int pivotLoc;
   // Base condition: stop when subarray has 0 or 1 element
   if(beg < end)
   {
      // Partition array and get correct pivot position
      partitionArray(a, beg, end, &pivotLoc);
```

```
        // Sort left subarray
        quickSort(a, beg, pivotLoc - 1);
        // Sort right subarray
        quickSort(a, pivotLoc + 1, end);
    }
} //end of quicksort() function

// Partition Function
void partitionArray(int *a, int beg, int end, int *pivotLoc)
{
    int left = beg;     // Left pointer starts from beginning
    int right = end;    // Right pointer starts from end
    *pivotLoc = left;   // Choose first element as pivot
    int tmp; // Temporary variable for swapping
    while(1)
    {
        // Move RIGHT pointer towards left
        // Stop when element smaller than pivot is found
        while(a[*pivotLoc] <= a[right] && *pivotLoc != right)
        {
            right--;
        }

        // If pivot reaches right, partition is complete
        if(*pivotLoc == right)
        {
            break;
        }
        // Swap pivot with smaller element
        else if(a[*pivotLoc] > a[right])
        {
```

```
            tmp = a[right];
            a[right] = a[*pivotLoc];
            a[*pivotLoc] = tmp;

            // Pivot shifts to right
            *pivotLoc = right;
        }
        // Move LEFT pointer towards right
        // Stop when element greater than pivot is found
        while(a[*pivotLoc] >= a[left] && *pivotLoc != left)
        {
            left++;
        }
        // If pivot reaches left, partition is complete
        if(*pivotLoc == left)
        {
            break;
        }
        // Swap pivot with larger element
        else if(a[*pivotLoc] < a[left])
        {
            tmp = a[left];
            a[left] = a[*pivotLoc];
            a[*pivotLoc] = tmp;

            // Pivot shifts to left
            *pivotLoc = left;
        }
    }//end of while(1) loop
    // At this point:
    // - Pivot is at correct sorted position
```

```
    // - Left side contains smaller elements
    // - Right side contains larger elements
}//end of partitionArray() function
```